



Fiche Projet – Pipeline PySpark de Traitement de Sinistres

Auteur : Souleymane Sow

Rôle : Data Engineer

Objectif : Implémenter un pipeline distribué pour le traitement de sinistres d'assurance : nettoyage, agrégation, détection de fraude, avec tests et CI/CD.



Technologies utilisées

- Python 3.11
- PySpark 3.5
- Pandas 2.3
- Pytest 8.4
- GitHub Actions (CI/CD)
- VSCode + Conda



Fonctionnalités principales

- Génération de données de test via CLI
- Pipeline PySpark modulaire et optimisé
- Détection de sinistres frauduleux
- Sauvegarde en Parquet
- Tests unitaires avec Pytest
- CI/CD GitHub Actions



Architecture du pipeline

1. **Lecture** du fichier Parquet (ou CSV, JSON, JDBC...)
2. **Nettoyage** des montants (correction des montants invalides par la médiane)
3. **Détection de fraude** via classes Python (`Sinistre`, `SinistreVol`)
4. **Agrégation** par client (`sum`, `avg`, `count`)
5. **Sauvegarde** des résultats en Parquet
6. **Tests unitaires** avec `pytest` (Python et Spark)
7. **CI/CD** avec GitHub Actions (exécution automatique des tests à chaque push ou pull request)
8. **Génération automatique** des données parquet de test via `scripts/generate_parquet.py`

Fichier .parquet de test généré

Le fichier `sinistres.parquet` est généré automatiquement via le script :

```
python scripts/generate_parquet.py
```

Il contient 1000 lignes simulées avec des montants valides, négatifs et manquants.

Résultat attendu

Un pipeline modulaire, testé, industrialisé, prêt à être intégré dans un environnement de production ou un projet d'équipe.

Installation rapide

- `git clone https://github.com/SouleymaneSow/sinistre-pipeline-python-pyspark.git`
- `cd sinistre-pipeline-python-pyspark`
- `conda create -n sinistre_env python=3.11`
- `conda activate sinistre_env`
- `pip install -r requirements.txt`
- `python -m scripts.generate_parquet`
- `python -m scripts.run_inference`
- `python -m pytest tests/`

Lancer les tests

Le projet inclut :

-  Tests techniques (Spark, schéma, colonnes)
-  Tests métier (règles métier, validation des données)
-  Tests de pipeline complet (`tests/test_pipeline.py`)

Exécution des tests :

- Tous les tests:
`python -m pytest tests/`
- Tests techniques uniquement:
 - 1) `make test-technique` ou
 - 2) `python -m tests/test_spark_validation.py`

- Tests métier uniquement:
 - 1) `make test-metier` ou
 - 2) `python -m tests/test_validation_metier.py`

Automatisation & Industrialisation

- Mise en place d'un **Makefile** pour centraliser toutes les commandes du projet
- Exécution rapide du pipeline (`make run`)
- Séparation claire des tests techniques (`make test-technique`) et métier (`make test-metier`)
- Génération automatique de la documentation (`make docs`)
- Démarche orientée **CI/CD et auditabilité**, facilitant la relecture et la reproductibilité

👉 Cela permet d'automatiser toutes les étapes (pipeline, tests, documentation) avec des commandes simples et reproductibles.

Qualité du code & CI/CD

- Mise en place de `pre-commit` pour automatiser le linting et les vérifications de style
- Intégration dans le **Makefile** via la commande `make lint`
- Assure une qualité de code homogène et reproductible

Optimisations Spark

- `cache()` pour éviter les recalculs
- `broadcast()` pour optimiser les jointures avec petites tables
- `repartition()` pour équilibrer les partitions
- Analyse du **Spark UI** pour identifier les shuffles et les goulets d'étranglement

Dépôt GitHub

👉 <https://github.com/SouleymaneSow/sinistre-pipeline-python-pyspark>

Lien la fiche projet PDF

👉 docs/fiche_sinistre_pipeline.pdf